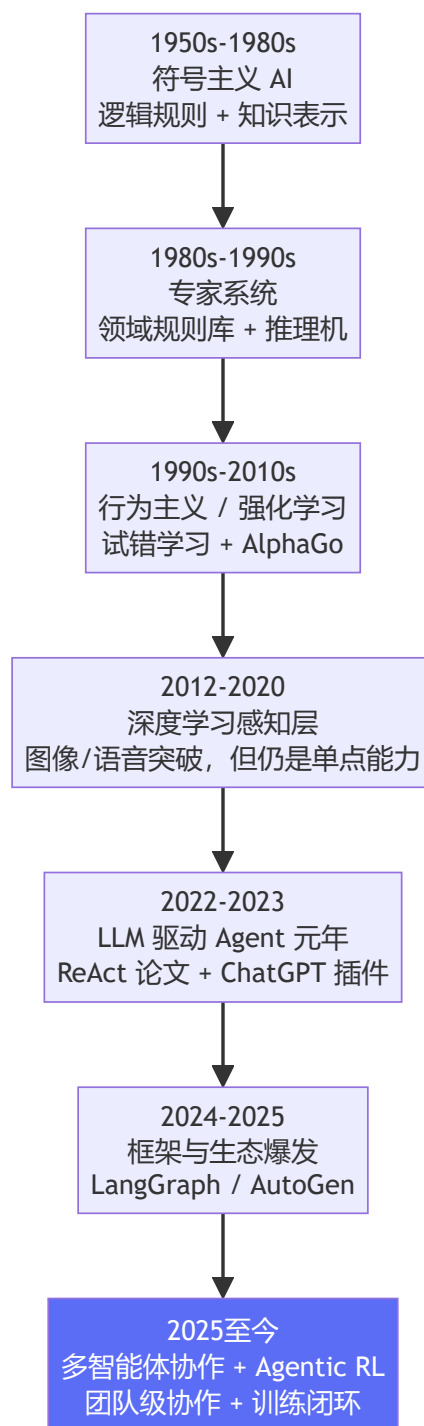


02 一张图看懂 Agent 进化史

为什么要学历史

产品经理经常问我："现在的 Agent 技术是不是又是一阵风？"要回答这个问题，最好的方式不是猜，而是看清楚：**过去 70 年里，"让机器自主做决策"这件事，已经经历过几轮兴衰，每一轮都是被同一个问题卡住的——机器没有足够的"常识"和"语言理解能力"去处理开放世界的模糊性。LLM 第一次同时解决了这两个问题，这才是这一轮不一样的地方。**

全景图



每个阶段到底发生了什么

###阶段一：符号主义 AI（1950s-1980s）—— "把世界写成逻辑规则"

核心思想：人类的智能可以被表达为符号和逻辑规则，机器只要有一套足够完整的规则库和推理引擎，就能"思考"。

代表：早期的自动定理证明、通用问题求解器（GPS）。

局限：现实世界的规则太多、太模糊、太容易有例外。"把苹果放进冰箱"这种常识对人类是废话，对符号系统却需要写几十条规则才能覆盖边界情况。这就是著名的"常识瓶颈"。

阶段二：专家系统 （1980s-1990s）—— "把规则收窄到一个 领域"

核心思想：既然通用规则太难写，那就把范围收窄到一个专业领域（医疗诊断、设备故障排查），由领域专家把经验写成规则库。

代表：MYCIN（医疗诊断专家系统）。

局限：规则库需要专家手工维护，规模一旦变大就"组合爆炸"——新增一条规则可能和已有几十条规则冲突，维护成本指数级上升。而且系统完全没有"举一反三"的能力，稍微超出规则库覆盖的场景就直接失灵。

阶段三：行为主义 / 强化学习（1990s-2010s）—— "让机器自己在环境里试错"

核心思想：不再靠人手写规则，而是让智能体在环境里反复试错，通过奖励信号自己学出策略。

代表：AlphaGo（2016）在围棋上击败人类顶尖棋手，是这条路线的巅峰。

局限：训练出来的策略高度依赖"环境定义得多干净"——围棋规则明确、奖励信号清晰，所以能训练成功。但换到"帮用户订一次机票"这种规则模糊、奖励信号难定义的开放任务上，强化学习几乎无法直接使用。这就是为什么AlphaGo 很强，但没有变成一个通用助手。

阶段四：深度学习感知层（2012-2020）—— "先让机器会看、会听"

核心思想：用深度神经网络解决感知层问题——图像识别、语音识别在这个阶段陆续超过人类水平。

局限：这些系统都是"单点能力"——识别猫和狗很强，但不会推理、不会规划、不能和你对话

着完成一个多步骤任务。感知强，认知弱。

阶段五：LLM 驱动 Agent 元年（2022-2023）—— "语言理解 + 世界知识 + 推理能力，三合一"

关键转折点：2022 年 ReAct (Reasoning + Acting) 论文提出，让 LLM 在生成回答的过程中交替进行"思考(Thought)"和"行动(Action)"，第一次让语言模型学会了"自己规划下一步该调用什么工具"。紧接着 ChatGPT 插件、AutoGPT 相继出现。

为什么这次不一样：前四个阶段分别单独解决了"逻辑规则"、"领域知识"、"试错学习"、"感知能力"中的某一项，但都缺"自然语言理解 + 常识推理"这个粘合剂。LLM 通过海量文本预训练，第一次同时具备了：

- 足够的世界常识（不用再手写规则）
- 自然语言理解与生成能力（能和人无缝交互）
- 一定程度的推理与规划能力（能拆解多步任务）

这三者叠加，才让"自主决策的智能体"第一次具备了通用性，而不再局限于围棋这种封闭环境。

阶段六：框架与生态爆发+ 多智能体协作 (2024-2025 至今)

LangGraph、AutoGen、AgentScope 等框架相继成熟，低代码平台（Dify/Coze/n8n）让 Agent 的搭建门槛大幅降低；同时，Agentic RL（用强化学习进一步训练 Agent 的决策能力，从 SFT 到 GRPO）开始把“试错学习”这条历史路线和 LLM 重新结合起来。这是我们现在所处的阶段。

给产品经理的启示：技术范式转移的规律

回看这条演进史，会发现一个反复出现的模式：

每一代范式的死因，都是“底层能力”跟不上“应用场景的开放程度”，而不是“这个方向本身错了”。

符号主义没有错，只是缺常识；专家系统没有错，只是缺泛化能力；强化学习没有错，只是缺清晰的奖励信号。LLM 的出现不是“取代”了前面几代，而是**补上了它们各自缺的那块拼图**——这也是为什么现在最强的 Agent 系统，往往是 LLM + 规则约束（符号主义的遗产）+ 强化学习微调（行为主义的遗产）的组合。

对产品经理的实际启示：**不要因为某个技术路线"过时"了就完全抛弃它的方法论。**专家系统的"领域规则库"思想，今天依然活在"给 Agent 设计护栏和工具白名单"里；强化学习的"奖励设计"思想，直接决定了你的 Agent 训练/评估该怎么打分。历史没有被推翻，是被拼接了。

产品经理清单

- ☐ 我的产品场景，是不是恰好落在了当前 LLM 能力的"能力边界"附近？（边界附近的场景，做出来的效果最不稳定）
- ☐ 有没有可能用"老范式"的方法论（规则约束、清晰的奖励/评估信号）去弥补纯 LLM 方案的短板？
- ☐ 我判断一个新技术"能不能用"，是看它解决了历史上哪个具体瓶颈，还是只是看它"新不新"？

下一章，我们把 LLM 这个"发动机"拆开来看——它到底强在哪、弱在哪，这些强弱怎么直接决定了你的 Agent 产品设计。

章节自测（5道单选，答案见文末）

Q1. LLM 驱动的 Agent 与之前几代范式相比，第一次同时解决了哪两个问题？ A. 算力和存储 B. 常

识/语言理解能力和自主决策能力
C. 成本和延迟 D. 训练和推理

Q2. ReAct 论文提出的关键突破是什么？ A. 让模型能训练 B. 让 LLM 在生成过程中交替进行思考和行动 C. 发明了向量检索 D. 提出了多智能体协作

Q3. 专家系统（1980s-1990s）最大的局限是什么？ A. 算力不够 B. 规则库维护成本随规模指数级上升，缺乏举一反三能力 C. 没有互联网 D. 没有 GPU

Q4. 强化学习/AlphaGo 这条路线为什么没能直接变成通用助手？ A. 算力不足 B. 依赖环境定义干净、奖励信号清晰，开放任务难以直接使用 C. 没有大模型 D. 成本太高

Q5. 文中给产品经理的核心启示是什么？ A. 只用最新技术 B. 每代范式死因是底层能力跟不上应用开放程度而非方向错了，老范式方法论依然有价值 C. 历史不重要 D. RL 已经过时

► [点击查看参考答案](#)